

Inspeccionar, construir y romper una cadena

Cincuenta minutos en clase; los minutos de cada ejercicio son un ritmo sugerido y el docente gestiona el tiempo, así que lo que no se cierre en el aula continúa como trabajo independiente. El lector de certificados y el simulador de la cadena de confianza corren en su navegador, sin conexión. Construir una autoridad y emitir certificados se hace con OpenSSL en su máquina: los comandos están abajo.

Antes de empezar. No se evalúa que los comandos corran. Se evalúa que expliquen **qué comprueba un navegador antes de confiar en un certificado, y en qué punto exacto falla el de Coopaburrá.** El hallazgo H5 se entiende cuando vean su propio navegador aceptar sin chistar un certificado que no debería.

¿En qué terminal? En Windows, los comandos de abajo van en **Git Bash** (viene con Git para Windows y trae OpenSSL y las órdenes head, tail y cat), **no en cmd ni en PowerShell**, donde no existen. En macOS o Linux, la Terminal normal. La alternativa en Python corre en cualquier terminal.

EJERCICIO 1 • 10 MINUTOS

Inspeccionar un certificado

Peguen un certificado en formato PEM y lean sus campos sin pasar por el navegador. Viene cargado uno de ejemplo con la estructura de la evidencia A del caso; reemplácelo por el de un sitio real.

CERTIFICADO EN PEM

```
-----BEGIN CERTIFICATE-----
MIIDpjCCAo6gAwIBAgIUWwzrsR7IbCWg4vWFC6WSgjSFCNwwDQYJKoZIhvcNAQEL
BQAwRDELMAkGA1UEBhMCQ08xEzARBgNVBAoMCKNvb3BhYnVycmExIDAEBgNVBAMM
F0Nvb3BhYnVycmEgUm9vdCBDQSAyMDE5MB4XDTE2MDkyMjIyMTkyMloXDTE3MTAy
NzIyMTkyMloQJELMAkGA1UEBhMCQ08xEzARBgNVBAoMCKNvb3BhYnVycmExIDAEBg
NVBAMMFxBvcnRhbnR5b29wYWJ1cnJhLnNvbTCCASIwDQYJKoZIhvcNAQEBBQAD
ggEPADCCAQoCggEBALHN3duSHtIH7Go0BPgJw6IiUDyfXXz42EZGFU1MGeE25ME
mJx0JwWPKBo/+V1P34zlx0jX0+ff+YjKElmtAhwvRkJFu1K4Sh21s2Gwx9H0Efig2
8h507MY10MURPZERqR1zbBQ1itc/+gZm1UkoUVMNjA0CpN+h30xvIEbrvdga//7+
vcN5nc3yGdFjbkYoIGt5gIcT8HnDtSc4y/SsEaaew3QN7oAbfBxmBJ25RW+78C3h
rLi09cWPTvF4Wrr76GRTsGmfSzOKiAiAUPYo9uTN+h88Xa0R5kNKUnNk0Cnd410I1
-----
```

Panel interactivo • ábralo en el navegador para ejecutarlo

PARA BAJAR EL CERTIFICADO DE UN SITIO REAL, DESDE LA TERMINAL

```
# deja el certificado del sitio en sitio.crt, listo para pegar arriba
echo | openssl s_client -connect www.iue.edu.co:443 -servername www.iue.edu.co 2>/dev/null | openssl x509
```

O EN PYTHON, QUE ES COMO VA EN EL INFORME

```
import ssl
from cryptography import x509

pem = ssl.get_server_certificate(("www.iue.edu.co", 443))
cert = x509.load_pem_x509_certificate(pem.encode())

print("Emisor: ", cert.issuer.rfc4514_string())
print("Vigencia:", cert.not_valid_before_utc, "->", cert.not_valid_after_utc)
nombres = cert.extensions.get_extension_for_class(x509.SubjectAlternativeName)
print("Ampara: ", nombres.value.get_values_for_type(x509.DNSName))
print("Firma: ", cert.signature_hash_algorithm.name)
```

Comparen con la evidencia A del caso. El certificado del portal no es débil en sí: RSA 2048 con SHA-256 es aceptable. El problema, que van a ver en la demostración siguiente, es **quién lo emitió** y qué pasa el 14 de junio de 2026.

DEMOSTRACIÓN • 5 MINUTOS

Qué comprueba el navegador

Antes de mostrar el candado, el navegador hace tres comprobaciones sobre el certificado. Cámbielas y vea cuándo confía y cuándo advierte. Coopaburrá falla en la primera.

LAS TRES COMPROBACIONES

- ☒ EL EMISOR ESTÁ EN EL ALMACÉN DE CONFIANZA DEL NAVEGADOR
- ☒ EL NOMBRE DEL CERTIFICADO COINCIDE CON EL DEL SITIO
- ☒ LA FECHA DE HOY ESTÁ DENTRO DE LA VIGENCIA

Panel interactivo • ábralo en el navegador para ejecutarlo

Por qué la primera comprobación es la que hunde a Coopaburrá

El navegador trae de fábrica una lista de autoridades certificadoras en las que confía: unas cuantas decenas, auditadas y públicas. Cuando llega un certificado, comprueba que lo haya firmado una de ellas, o alguien en quien una de ellas confíe. La autoridad interna de Coopaburrá, «Coopaburra Root CA 2019», no está en esa lista y nunca lo estará: la creó un contratista en 2019 para no pagar certificados.

Por eso los asociados ven una advertencia en cada visita. Y como llevan años viéndola, aprendieron a aceptarla sin leerla —exactamente lo que un atacante necesita—. El ejercicio 3 les va a mostrar por qué esa advertencia no es un formalismo.

EJERCICIO 2 • 15 MINUTOS

Construir su propia autoridad

Creen una raíz, emitan un certificado de servidor y háganlo funcionar. Van a reproducir exactamente lo que hizo Coopaburrá en 2019, y a sentir por qué es tan fácil.

```
# 1 • crear la raíz: su propia autoridad certificadora
openssl req -x509 -newkey rsa:4096 -keyout raiz.key -out raiz.crt -days 3650 \
  -nodes -subj "/CN=Mí Autoridad de Práctica"

# 2 • una llave y una solicitud para un servidor
openssl req -newkey rsa:2048 -keyout servidor.key -out servidor.csr \
  -nodes -subj "/CN=mí-sitio.local"

# 3 • la raíz firma el certificado del servidor
openssl x509 -req -in servidor.csr -CA raiz.crt -CAkey raiz.key \
  -CAcreateserial -out servidor.crt -days 365

# 4 • verificar la cadena
openssl verify -CAfile raiz.crt servidor.crt
```

Cuatro comandos. Eso es todo lo que hace falta para montar una autoridad certificadora que emite certificados de apariencia impecable. La dificultad no está en crearla: está en custodiar la llave de la raíz, y ahí es donde Coopaburrá falló. Esa llave está en un computador encendido, sin contraseña, de alguien que ya no trabaja allí.

QUÉ SE ENTREGA DE ESTE EJERCICIO

- La salida de **openssl verify** confirmando la cadena.
- Una frase: si crear la autoridad es tan fácil, ¿qué es exactamente lo difícil, y por qué existen las autoridades públicas?

EJERCICIO 3 • 15 MINUTOS

Romper la cadena de confianza

El ejercicio que hay que entender. Con esa misma raíz, emitan un certificado para un dominio que **no es suyo**, instalen la raíz en su navegador y observen que no aparece ninguna advertencia.

```
# emitir un certificado para un dominio ajeno, con SU raíz
openssl req -newkey rsa:2048 -keyout falso.key -out falso.csr \
  -nodes -subj "/CN=www.un-banco-cualquiera.com"
openssl x509 -req -in falso.csr -CA raiz.crt -CAkey raiz.key \
  -CAcreateserial -out falso.crt -days 365

# mirar el certificado: dice ser de un dominio que no es suyo
openssl x509 -in falso.crt -noout -subject
```

Háganlo solo en su equipo y retiren la raíz al terminar. Cuando instalen su raíz en el navegador y este acepte sin chistar un certificado que ustedes emitieron para un dominio ajeno, van a entender el hallazgo H5 mejor que con cualquier explicación. Lo único que separa a un impostor de un sitio legítimo es **quién controla una raíz de confianza**. Quien tenga la llave de la raíz de Coopaburrá puede hacer esto contra cualquier sitio, para los 142.000 asociados.

QUÉ SE ENTREGA DE ESTE EJERCICIO

- El certificado falso, con el **subject** que muestra un dominio que no es suyo.
- La explicación de por qué el navegador lo aceptó, ligada a la primera comprobación de la demostración.
- **La frase clave:** qué puede hacer, concretamente, quien tenga la llave privada de la autoridad interna de Coopaburrá, que está en el computador del ex contratista.

EJERCICIO 4 • 10 MINUTOS

Diagnosticar el canal

Revisen las versiones y las suites que ofrece un servidor, y produzcan el listado de lo que habría que apagar, con la justificación de cada línea. La evidencia A dice que el portal ofrece TLS 1.0, 1.1 y 1.2.

```
# versiones y suites que acepta un servidor
nmap --script ssl-enum-ciphers -p 443 www.iue.edu.co

# o, sin nmap, probar una version concreta:
openssl s_client -connect www.iue.edu.co:443 -tls1_2 </dev/null
openssl s_client -connect www.iue.edu.co:443 -tls1_3 </dev/null
```

QUÉ SE ENTREGA DE ESTE EJERCICIO

- El listado de versiones que habría que desactivar en el portal, según la evidencia A, con una razón por cada una.
- Qué versión debería quedar como mínimo, y por qué las anteriores ya no se aceptan en 2026.

¿Qué comprobación falló?

Seis situaciones sobre certificados y canal. Elijan la causa.

1. El navegador muestra una advertencia a pantalla completa en cada visita al portal, aunque el certificado no está vencido.

2. Un certificado es técnicamente RSA 2048 con SHA-256, pero su llave privada está en un equipo sin contraseña de alguien que ya no trabaja en la empresa.

3. Alguien emite, con una raíz que controla, un certificado a nombre de un banco que no es suyo, y un navegador con esa raíz instalada lo acepta.

4. El servidor todavía acepta conexiones por TLS 1.0 y TLS 1.1.

5. El certificado del portal vence el 14 de junio y nadie sabe quién lo renueva.

6. Los asociados aprendieron a hacer clic en 'aceptar el riesgo' cada vez que entran al portal.

CIERRE

Qué se entrega y cómo se califica

Una página por equipo al final del bloque. Se evalúa el análisis, no el resultado.

- ☐ Los campos del certificado del ejercicio 1, con la comparación frente a la evidencia A.
- ☐ La salida de **openssl verify** del ejercicio 2 y la frase sobre qué es lo difícil de una autoridad.
- ☐ El certificado falso del ejercicio 3 y la explicación de por qué el navegador lo aceptó.
- ☐ El listado de versiones a desactivar del ejercicio 4, con su justificación.
- ☐ Una frase que conecte todo: por qué el hallazgo H5 es, a la vez, un problema de custodia de llave y un problema de confianza.

Regla de trabajo del módulo. La raíz de práctica que instalen en el navegador es un riesgo mientras esté instalada: cualquiera con su llave podría suplantar sitios ante su equipo. Retírenla apenas terminen el ejercicio 3.

